

Personalized Email Prioritization System

Project Proposal for UCS503P

Submitted to: Dr. Raghav B. Venkataramaiyer
Thapar Institute of Engineering and Technology

Aryan Sharma, CSED¹

Maithli Arora, CSED²

Tushar Sharma, CSED³

August 17, 2026

1 Higher-order goal

This project aims to improve personal digital productivity by reducing the time and attention that individuals lose to email overload. While the problem of information overload is broader than any single inbox, the immediate engineering objective is to translate the idea of a self-prioritizing inbox into a measurable, deployable software solution that learns what matters to each user. We refer to this system as *Vigilante*.

2 Time-to-value

- We will deliver meaningful increments quickly by prototyping the core scoring-and-labelling workflow first and validating it against real inboxes within a short iteration window.
- We will prioritize fast feedback over perfection by using weekly iterations (and targeting CI-backed automation early) to reduce release friction.
- Success is defined by what can be delivered and measured in the first iteration — a working classifier applied to a connected inbox — and then improved based on user feedback.

3 Problem Statement

Modern email users receive far more mail than they can meaningfully attend to. A typical inbox is dominated by newsletters, promotions, and automated notifications, while the genuinely important messages — a security alert, a payment notice, a reply from a person, a deadline — are a small minority buried within the noise. Common pain points include:

- **Attention overload:** important messages are lost among a high volume of low-value mail.
- **One-size-fits-all filters:** existing rules (e.g., “Promotions” tabs) are generic and cannot learn an individual’s personal notion of importance.
- **Static rules break on real mail:** a security code and a marketing blast can arrive from the same kind of no-reply address, so sender-only rules mislabel critical mail.
- **No personalization over time:** conventional filters do not adapt as a user’s priorities and correspondents change.

Why this matters now (contextual relevance): As inbox volume grows, even a modest, personalized reduction in triage effort meaningfully reclaims user attention and reduces the risk of missing time-sensitive mail.

¹Roll No: 1024030679, Email: asharma22_be24@thapar.edu

²Roll No: 1024030687, Email: marora1_be24@thapar.edu

³Roll No: 1024030676, Email: tsharma3_be24@thapar.edu

4 Proposed Solution

4.1 Overview

Build a web-based “Inbox-to-Priority” system, *Vigilante*, with the following components:

- **Secure inbox connection:** users connect a Gmail account through a read-only OAuth 2.0 flow; the system never receives the account password.
- **Importance classifier:** a machine-learning model scores each email as important or unimportant from its metadata (sender, subject, headers).
- **Personalization layer:** an online-learning component adapts the scoring to each user’s behaviour over time.
- **Priority view and labelling:** important mail is surfaced; low-value mail is de-emphasized, with clear, reversible labels.
- **Admin/ops dashboard:** summaries of classification volume, confidence, and model behaviour.

4.2 Core workflow

1. User connects their inbox via OAuth (read-only) and the system fetches recent email metadata.
2. The baseline model scores each email and assigns an initial important/unimportant label.
3. The user’s interactions (opens, replies, corrections) feed the personalization layer.
4. The personalization layer adjusts future scores so the inbox increasingly reflects the user’s own notion of importance.

4.3 Operational constraints for an educational setting

- Keep the solution usable on low-to-mid range devices via responsive web design.
- Use read-only access and minimal data retention to respect user privacy.
- Rely on standard, well-understood models (logistic regression, TF-IDF) rather than novel research algorithms; focus on workflow, personalization, and system correctness.
- Design for deployability using common web hosting and managed databases.

5 Solution Approach

This is an engineering course project, so we focus on the scalable classification and personalization workflow rather than novel algorithm research.

5.1 Classification and personalization

The importance model will be built in two layers:

- **Baseline classifier:** a logistic-regression model over engineered features (sender type, unsubscribe headers, reply markers, subject length) combined with TF-IDF features on subject lines, trained on real email data. This provides a sensible cold-start judgement for a new user.
- **Personalization layer:** a lightweight online-learning component (contextual bandit) that adjusts scores per user from their interactions, capturing the personal notion of importance a static model cannot.

Labels for training are derived using an intent-based method (transactional and personal mail treated as important; marketing, newsletters, and promotions as unimportant), without claiming state-of-the-art.

5.2 Web architecture

A typical 3-tier web architecture:

- **Frontend:** responsive UI for connecting an inbox, viewing the prioritized inbox, and correcting labels.
- **Backend API:** OAuth handling, email fetching, a scoring service that runs the model, and storage of user feedback.
- **Database:** store users, tokens (securely), classification results, and interaction history.

5.3 CI/CD and fast delivery enabler

CI/CD will be used to:

- Automatically build and run tests on every change.
- Produce repeatable deployment artifacts to staging.
- Enable safe and frequent releases of incremental features.

6 Evaluation Criterion

We define success using measurable metrics tied to the core classification workflow.

6.1 Primary evaluation metric

Important-class recall: the proportion of genuinely important emails that the system correctly surfaces.

- **Target:** important-class recall ≥ 0.85 on a held-out, human-judged sample during the pilot window.
- **Attribution:** measured against user-labelled ground truth on a connected inbox.

6.2 Secondary evaluation metrics

- **Precision on important mail:** minimize false alarms (unimportant mail surfaced as important).
- **Personalization gain:** improvement in recall/precision after the personalization layer adapts to a user, compared to the baseline alone.
- **User clarity:** user feedback on whether the priority view is helpful, using a simple 1–5 question.
- **Reliability:** availability of the scoring service and inbox connection during the pilot (e.g., $\geq 99\%$ uptime).

6.3 Pilot validation plan

- Recruit 10–20 pilot users who connect their own inboxes (with consent, read-only).
- Have each user label a sample of their mail as ground truth, and compare model output against it before and after personalization within the iteration window.

7 Scalability

The proposition should scale in both theoretical and practical senses.

1. **Scalable (theoretically):** the system should support growth in number of connected inboxes and classification volume by:
 - decoupling components (frontend / backend / scoring service),
 - batching and indexing email-classification queries,
 - using a stateless API design with per-user model state stored separately.
2. **Operationally deployable (practically):** the system should run on common web hosting:
 - managed database,
 - containerized or platform-hosted deployment,
 - CI/CD pipeline for staging / production.

8 Engine availability

A mature set of “engines” (tooling / services) is available to implement this as a web-based project:

- Web framework for REST APIs and reactive pages.
- Managed relational or document database.
- Gmail API with OAuth 2.0 for secure, read-only inbox access.
- A machine-learning toolkit (scikit-learn) for the baseline classifier and vectorizer.
- Test framework for unit / integration tests.
- CI runner and staging deployment target.

We will use these mature components to focus on classification workflow, personalization, correctness, and measurement rather than inventing new infrastructure.

9 Project scope and deliverables

9.1 Initial deliverable

- Read-only OAuth inbox connection and email-metadata fetching.
- Baseline classifier applied to a connected inbox, producing important/unimportant labels.
- Priority view: the inbox sorted/labelled by importance.
- Basic logging and metrics for recall/precision measurement.
- CI: build + backend unit tests + linting.
- CD to staging with a single command or pipeline job.

9.2 Subsequent deliverables

- Personalization layer that adapts scores from user feedback.
- Notification / digest layer (optional in a later iteration).
- Admin dashboard for classification metrics and model behaviour.
- Improved feature set and indexed queries for scale readiness.

10 Risks and mitigations

- **Data privacy / consent:** use read-only access, store minimal metadata, and never commit or expose user credentials or inbox contents.
- **User adoption:** keep the priority view simple and all labels reversible so users retain trust and control.
- **Evaluation measurement ambiguity:** define important/unimportant labelling criteria and ground-truth collection before development begins.
- **Model limitations:** a static baseline cannot judge every case; mitigate with the personalization layer and clear, correctable labels.

11 Summary

This project proposes *Vigilante*, a deployable web platform that reduces email-triage effort by learning each user’s personal notion of importance. It meets the course criteria by emphasizing time-to-value through rapid iterations, implementing CI/CD to support frequent safe releases, and using measurable evaluation criteria (especially important-class recall). It remains focused on engineering workflow, personalization, and scalability readiness rather than research-driven novelty.